

综合实践 III 课程设计报告

——单目标跟踪系统的设计与实现——

学 院 计算机科学与工程学院

专 业 计算机科学与技术

班 级 【班级】

学生姓名 【姓名】

学 号 【学号】

指导老师 罗颂、罗娅、贺筠

时 间 2026 年 7 月

课程设计评分表

课程设计报告评分依据	成绩占比	成绩
课程目标 3：能够运用科学方法，对计算机领域的复杂工程问题进行需求和功能分析。 评价标准：具有多角度全面性的局限性和可行性分析。对照多种方案进行方案优选或改进。	50%	
课程目标 5：能够理解多学科背景下的团队中每个角色的定位与责任，能够与团队的成员进行有效沟通，并胜任个人承担的角色任务。 评价标准：有规范、详细、完整的项目分析，项目功能设计完善。	25%	
课程目标 6：能够实施团队工作的过程管理，针对计算机应用系统的分析、设计、开发、测试等环节制定、实施工作计划，能够胜任负责人的角色。 评价标准：有明确的小组分工，有完整的项目管理过程和管理证明材料。	12.5%	
课程目标 7：理解多学科复杂工程问题的知识融合理念，能够组织和管理计算机软硬件项目，并在项目设计、开发等工	12.5%	

程实践中，认知或实施质量检测、安全保障，认知复杂系统安全预案的必要性。 评价标准：有完整的项目测试记录及对应的测试结果分析，有全面客观的项目质量评定。		
合计		
评阅教师（签名或签章）		

团队成员

本项目以两人团队方式完成，团队所有成员信息如下（团队角色为组长或成员，整个团队不超过 2 人）：

学号	姓名	团队角色	团队分工
【学号 1】	【姓名 1·本人】	组长	视觉跟踪算法设计实现 (Python+OpenCV)； Web 后端设计与实现 (Spring Boot)；三端 集成与系统测试；报 告撰写
【学号 2】	【姓名 2】	成员	微信小程序端设计与 实现；Web 管理端页 面设计与实现；数据 库设计；测试用例编 写与答辩 PPT

注：请打印前删除本行说明，并按实际情况填写上方学号、姓名与分工。

摘 要

针对课程设计提出的“小程序端提交视频、服务端完成视觉处理、Web 端进行管理”的三端联动应用场景，本文设计并实现了一个基于 OpenCV 的智能单目标跟踪系统。系统以微信小程序为用户端，提供用户注册登录、视频上传与处理结果回看等功能；以 Spring Boot + MyBatis 为 Web 后端，负责用户认证（JWT）、任务调度与文件存储，并为管理员提供基于浏览器的 Web 管理端，实现对用户的增删改查、任务记录查询与系统统计；以 Python + OpenCV 为核心视觉处理层，采用 CSRT 相关滤波跟踪器实现目标的持续跟踪，并针对目标被遮挡后丢失的问题，设计了“直方图质量校验 + 多尺度模板匹配 + 连续帧确认”的遮挡检测与自动重检测恢复机制：当目标被短暂遮挡后重新出现时，系统能够再次捕捉目标并继续跟踪。经功能测试与遮挡恢复专项测试，系统各模块功能正确，在 640×480 分辨率视频上处理速度约 95~101 帧/秒，完全遮挡恢复场景中目标丢失 1 次并成功重新捕获 1 次，验证了系统设计的可行性与稳定性。

本文遵循软件工程规范，从系统目标与分析、项目管理、总体设计与详细设计、系统实现、系统测试与总结等方面，完整记录了项目的分析、设计、实现、测试与总结全过程。

关键词：单目标跟踪；OpenCV；遮挡重检测；微信小程序；Spring Boot

课程设计评分表	2
团队成员	3
1 系统目标与分析	7
1.1 系统目标	7
1.2 系统分析	8
1.3 关键问题及解决办法	11
2 项目管理	13
2.1 组员分工	13
2.2 项目进度控制	13
2.3 质量保障	14

2.4 系统集成	15
3 单目标跟踪系统的设计与实现	17
3.1 总体设计	17
3.2 详细设计	19
3.3 系统实现	25
4 系统测试	36
4.1 测试环境	36
4.2 测试记录	36
4.3 测试结论及分析	38
4.4 测试结论	38
5 系统总结	39
5.1 系统特色	39
5.2 展望工作	39
5.3 使用说明	39
6 心得体会	41
参考文献	42
附录 A 项目源代码目录结构	43

1 系统目标与分析

1.1 系统目标

本课程是计算机科学与技术专业的专业实践类课程，课程设计的目的是锻炼学生解决复杂工程问题的能力，使学生通过完成指定或自选的项目践行软件工程思想，经历完整的问题分析、设计、实现、测试以及项目实施过程管理与总结的循环。根据《综合实践 III》任务书要求，课题需同时涉及 Web 开发（管理端）、移动开发（小程序端）与机器视觉（核心视觉处理）三个方面，系统流程为：用户从小程序端登录，提交要处理的图片或视频，服务器端的视觉处理模块对素材进行处理并返回结果，提交素材与处理结果保存至数据库，管理员从 Web 管理端登录后可对用户进行增删改查等管理操作。

本项目选择任务书视觉场景中的“单目标跟踪”方向，确定系统目标如下：

（1）完成一套三端联动的单目标跟踪系统：微信小程序端面向普通用户，提供注册、登录、修改密码、上传跟踪视频、查看处理结果与历史记录功能；

（2）Web 管理端面向管理员，提供用户查询、新增、删除、重置密码，以及任务记录查询、删除和系统统计功能；

（3）服务器端视觉处理模块基于 Python + OpenCV 实现单目标跟踪：用户指定视频首帧中的目标对象后，系统持续跟踪并绘制其运动轨迹；当目标被短暂遮挡后再次出现在画面中时，系统能依据目标特征重新捕捉目标并继续跟踪，即具备“遮挡丢失—自动重检测—恢复跟踪”能力；

（4）提交的视频与跟踪结果（结果视频、轨迹日志、统计信息）统一存储于数据库与文件系统，支持按用户查询回看；

（5）全程遵循软件工程规范：界面展示完整、命名与注释规范、使用 Git 进行团队版本控制、编写规范的文档与测试记录。

1.2 系统分析

1.2.1 可行性分析

技术可行性：单目标跟踪是机器视觉领域的经典问题。OpenCV 提供了成熟的 CSRT、KCF 等相关滤波跟踪器实现，不需要深度学习训练过程，一台普通 PC 即可达到实时处理速度；微信小程序开发生态成熟，Spring Boot + MyBatis + MySQL 是目前 Web 开发的主流技术栈，本机已具备 Java 17、Maven、Python 3.13（含 OpenCV 4.10）与 MySQL 8.0 环境，开发与运行环境齐备，技术路线完全可行。

经济可行性：本项目全部采用免费开源组件（OpenCV、Spring Boot、MySQL、微信开发者工具等），无额外软件采购成本；所需的开发设备为普通个人计算机，经济上完全可行。

操作可行性：小程序端与 Web 管理端界面均采用简洁、易操作的设计，普通用户只需完成选择视频与提交两步操作即可获得跟踪结果，管理员管理操作均有明确提示，系统易学易用，操作上可行。

1.2.2 功能需求分析

依据任务书对“用户管理、视觉处理、图片/视频的存储和查询”的功能要求，结合单目标跟踪场景，系统功能需求如表 1-1 所示。

表 1-1 系统功能需求列表

功能模块	编号	功能描述	角色
用户注册	F-01	填写用户名/密码/昵称完成注册，用户名唯一，密码长度不少于 6 位	普通用户
用户登录	F-02	用户名+密码登录，签发 JWT 令牌	普通用户/管理员
修改密码	F-03	校验原密码后修改密码	普通用户
视频上传	F-04	选择本地视频上传，可指定首帧目标框或自动选择	普通用户
跟踪处理	F-05	服务端视觉模块执行单目标跟踪与遮挡重检测	系统

结果回看	F-06	查询任务状态、播放结果视频、查看跟踪统计	普通用户
历史记录	F-07	分页查看本人历史任务记录	普通用户
用户管理	F-08	用户查询/新增/删除/重置密码	管理员
任务管理	F-09	任务记录查询（按状态过滤）/删除	管理员
系统统计	F-10	用户数、任务数及处理状态统计	管理员

系统总体用例图如图 1-1 所示，普通用户可进行注册、登录、修改密码、上传视频、查看处理结果与历史记录等操作；管理员在上述基础上还可进行用户管理、任务记录管理与系统统计。



图 1-1 系统总体用例图

结合任务书给定的系统流程（用户登录 → 提交视频/图片 → 服务器端视觉处理 → 返回结果 → 结果入库 → 管理端管理），系统整体处理时序如图 1-2 所示：用户上传视频后后端异步调度视觉服务，视觉服务完成跟踪后将结果回传后端并入库，前端通过轮询获取最终结果。

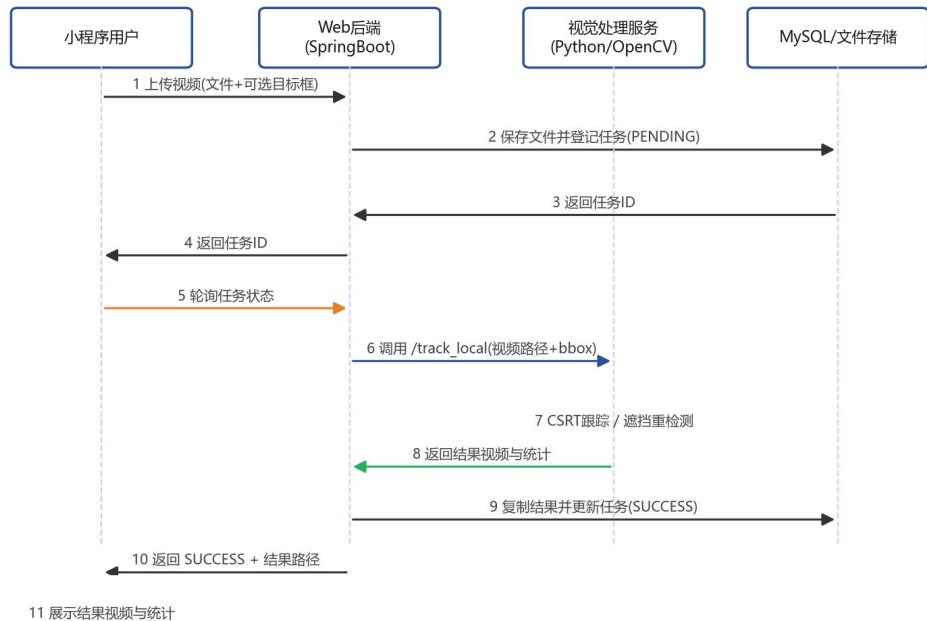


图 1-2 系统处理时序图

以核心的“视频上传与跟踪处理”为例，其用例描述如表 1-2 所示（参照报告模板表 1.1 格式）。

表 1-2 “视频上传与跟踪处理”用例描述

用例名：	视频上传与跟踪处理
标识号：	UC-001
参与者：	普通用户
用例描述：	用户选择一段视频并指定（或自动选择）首帧目标对象，提交后由服务端视觉模块完成单目标跟踪，返回带跟踪框的结果视频与跟踪统计信息。
前置条件：	用户已登录系统。
后置条件：	任务状态更新为 SUCCESS（或 FAILED），结果文件已存储并生成访问路径。
基本操作流程：	① 用户在小程序端登录；② 选择要跟踪的视频；③ 选择“自动目标”或输入首帧目标框 bbox；④ 点击提交，系统返回任务 ID；⑤ 视觉模块异步处理，前端轮询任务状态；⑥ 处理

	完成后展示结果视频与“丢失/重捕获”统计。
备选操作流程:	若视频格式不支持或目标框非法，系统提示上传失败；若目标全程丢失未能找回，任务状态置 FAILED 并记录原因。

1.2.3 非功能需求分析

性能需求：视觉模块对 640×480 分辨率视频的处理速度应达到实时或准实时（不低于 25 帧/秒）；上传接口对视频文件大小限制为 200MB。安全性需求：用户密码使用 BCrypt 加盐哈希存储，业务接口统一使用 JWT 令牌鉴权，管理端接口需要 ADMIN 角色权限。可靠性需求：视觉处理采用异步执行与状态机管理

（PENDING→PROCESSING→SUCCESS/FAILED），任一环节失败均有明确错误记录，不阻塞用户其他操作。易用性需求：小程序与管理端界面层级不超过三层，关键操作均有提示与确认。

1.3 关键问题及解决办法

本系统开发中涉及的关键问题及其解决办法分析如下：

表 1-3 系统关键问题及解决办法

关键问题	问题分析	解决办法
跟踪算法选型与实时性	需兼顾跟踪精度与处理速度，任务书要求“基于视频流实时跟踪”	对比 KCF/CSRT 等主流跟踪器：KCF 速度快但尺度不变性差，CSRT 精度高、支持尺度变化。本系统选用 CSRT 为主跟踪器并支持 KCF 切换，实测约 95~101 帧/秒，满足实时性要求。
目标被遮挡后的再捕捉	遮挡时相关滤波模板被背景污染，直接恢复跟踪会丢失目标	设计“TRACKING/SEARCHING”双状态机：跟踪期间周期性用 HSV 直方图相关度与越界检查判断是否丢失；丢失后进入搜索模式，以初始参考模板进行多尺度匹配并用直方图验证，连续 2 帧命中即重新初始化跟踪器，

		实现再捕捉。
视频大文件上传 与长任务处理	视频文件可达数十~数百 MB，视觉处理耗时长， 同步接口易超时	后端提供 multipart 上传接口并异步线程池调度视觉处理；前端以任务 ID 轮询状态，处理完成后再拉取结果，避免长连接阻塞。
三端数据流与结果共享	小程序、Web、视觉服务之间数据格式与文件访问方式不统一	统一 REST + JSON 契约；结果文件统一落盘到后端存储目录，通过 /files/** 静态映射对外提供 URL，三端只需访问统一地址。
任务失败的可见性	视觉服务异常、文件损坏等原因会导致任务中断	任务表记录 error_msg 与 finish_time ，状态置 FAILED ；管理端与小程序均可查看失败原因，便于定位与重试。

2 项目管理

2.1 组员分工

本项目由 2 人组队完成，采用 Git 作为版本控制工具进行协作。为降低模块间耦合、便于并行开发，按“三端 + 数据”划分工作包，分工如表 2-1 所示。

表 2-1 团队成员分工表

成员	团队角色	主要职责	对应代码目录
【姓名 1·本人】	组长	视觉跟踪模块 (Python+OpenCV) 设计实现与算法调 优；Web 后端 (Spring Boot) 设计 与实现；三端接口联 调与系统测试	01-代码/vision、01-代 码/server
【姓名 2】	成员	微信小程序端全部页 面与交互实现；Web 管理端页面实现；数 据库表结构设计与测 试数据构造	01-代码 /miniprogram、 server/static

协作过程说明：开发前共同编写接口约定文档（REST 路径、参数、返回格式）；开发中以 Git 分支进行功能开发，各自完成后合并到主分支并解决冲突；联调阶段统一在本地环境按“视觉服务 → 后端 → 小程序/管理端”顺序启动，逐接口验证。

2.2 项目进度控制

项目周期共安排 8 周，各阶段计划与时间安排如表 2-2、图 2-1 所示。开发过程中按里程碑（视觉模块可运行、后端接口可调用、三端全流程贯通、测试完成）进行进度检查，实际进度与计划基本一致。

表 2-2 项目进度计划表

阶段	起止时间	主要工作	里程碑产出
第 1 周	第 1 周	需求分析、方案设计、技术调研	设计文档、任务分解
第 2 周	第 2 周	环境搭建、数据库设计	建库脚本、工程骨架
第 3~4 周	第 3~4 周	视觉跟踪模块开发（CSRT+遮挡重检测）	可运行跟踪器与测试视频
第 5 周	第 5 周	Web 后端与管理端开发	REST 接口、管理页面
第 6 周	第 6 周	小程序端开发	小程序全部页面
第 7 周	第 7 周	三端集成联调、缺陷修复	全流程贯通
第 8 周	第 8 周	系统测试、报告撰写、答辩准备	测试记录、报告、PPT

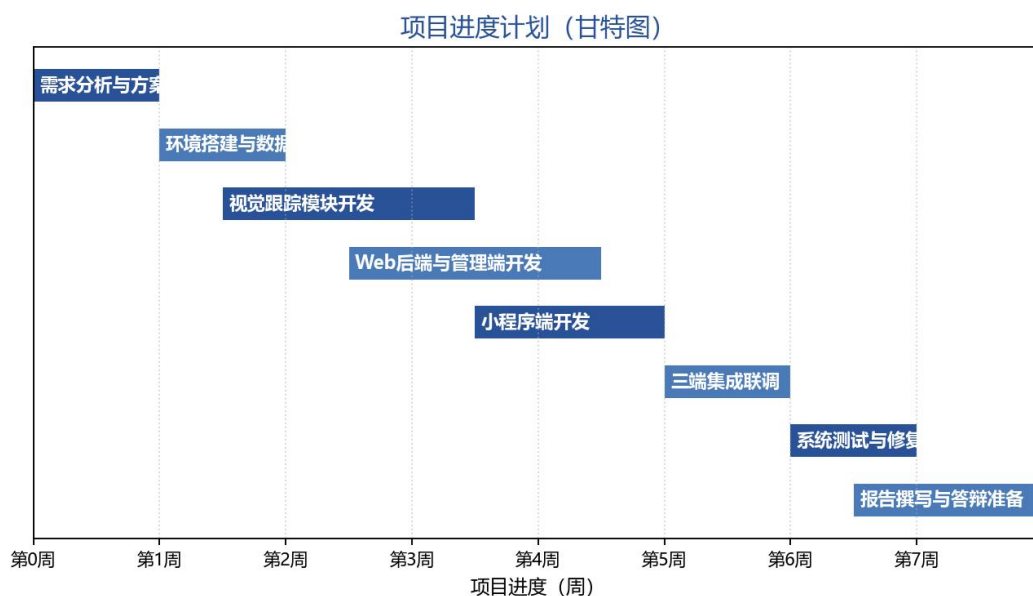


图 2-1 项目进度甘特图

2.3 质量保障

为保证代码与文档质量，项目组制定了以下质量保障措施：

（1）编码规范：类名、方法名、变量名采用规范命名（如 SingleObjectTracker、processVideo）；关键逻辑添加注释；每个源文件头部注明创建时间、作者（学号姓名）与功能描述，符合任务书“在适当的位置添加注释，反映自己的学号姓名”的要求；

(2) 版本控制：使用 Git 管理全部代码，按功能提交并书写规范提交信息，典型提交记录如表 2-3 所示；

表 2-3 Git 提交记录（示例，以实际仓库为准）

提交说明	涉及模块	提交人
feat(vision): CSRT 跟踪器与遮挡重检测状态机	vision/tracker.py	【姓名 1】
feat(server): 认证与用户管理接口 (JWT+BCrypt)	server 后端	【姓名 1】
feat(mini): 上传跟踪与结果轮询页面	miniprogram	【姓名 2】
feat(admin): Web 管理端用户/任务管理页面	server/static	【姓名 2】
fix(vision): 完全遮挡场景无法触发丢失的缺陷	vision/make_demo_video.py	【姓名 1】
test: 全链路功能测试与测试记录	demo、文档	【姓名 1】【姓名 2】

(3) 评审与自查：每个模块完成后由对方交叉检查代码（命名、异常处理、接口一致性），联调发现的缺陷统一登记并修复；

(4) 文档保障：按模板格式同步撰写课程设计报告，测试过程如实记录，保证“代码—测试—文档”三者一致。

2.4 系统集成

系统集成按“自底向上”策略分三步进行：第一步，单独验证视觉模块（命令行/接口方式对标准测试视频完成跟踪并输出结果）；第二步，验证后端与视觉服务的接口对接（track_local），以及后端与 MySQL 的数据读写；第三步，完成“小程序→后端→视觉服务→数据库→管理端”的全链路串联，冒烟测试结果如表 2-4 所示。

表 2-4 系统集成冒烟测试记录

集成项	验证方式	结果
视觉服务自检	GET /api/v1/health 返回 ok	通过
视觉处理接口	POST /api/v1/track_local 对测试视频处理成功	通过

后端-数据库	注册/登录后查询用户信息一致	通过
上传全链路	小程序模拟上传 → 状态轮询至 SUCCESS → 结果视频可访问	通过
管理端接口	admin 登录后用户/任务/统计接口正常	通过
结果文件服务	GET /files/results/*.mp4 播放正常	通过

3 单目标跟踪系统的设计与实现

3.1 总体设计

3.1.1 设计思路与总体架构

依据任务书对系统流程的要求，本系统按“表现层—业务服务层—视觉处理层—数据层”四层架构进行设计，总体架构如图 3-1 所示。

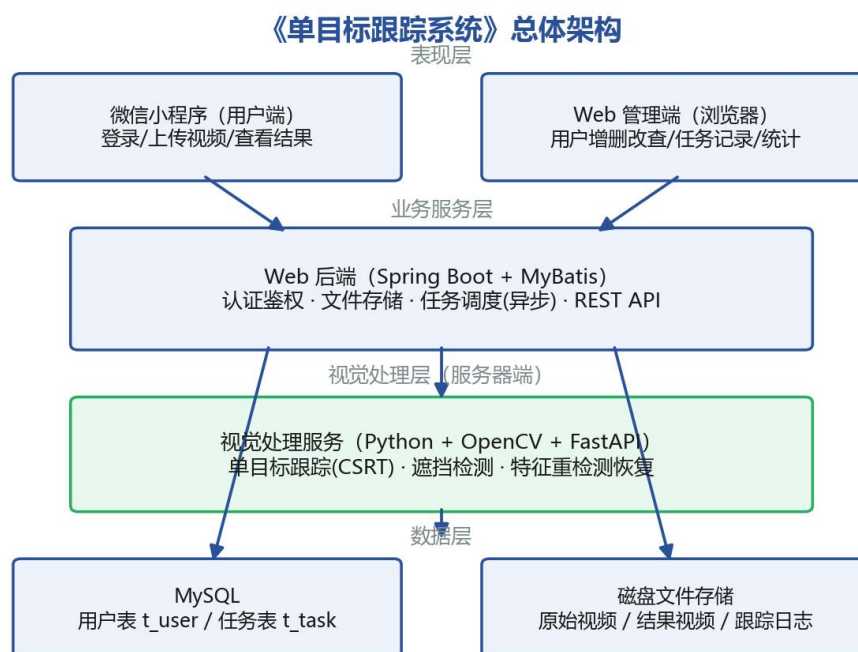


图 3-1 系统总体架构图

各层职责如下：表现层包括微信小程序（普通用户）与 Web 管理端（管理员）；业务服务层为 Spring Boot Web 后端，承担用户认证、文件存储、视觉任务调度与 REST API 服务；视觉处理层为独立的 Python + OpenCV 服务，通过 FastAPI 对外提供单目标跟踪接口；数据层使用 MySQL 保存用户与任务信息，使用磁盘文件系统保存原始视频、结果视频与跟踪日志。视觉处理层独立部署，通过 HTTP 接口与后端解耦，便于后续替换或升级算法。

3.1.2 技术选型

表 3-1 关键技术选型及理由

层次	技术	选型理由
用户端	微信原生小程序	任务书指定小程序端；无需安装、生态成熟，wx.uploadFile 天然支持文件上传
管理端	原生 HTML+CSS+JS 单页	零构建、零依赖，随后端静态资源发布，答辩演示环境适应性强
后端	Spring Boot 3 + MyBatis + JWT + BCrypt	主流企业级组合，注解式 SQL 开发高效；JWT 无状态鉴权、BCrypt 密码安全
数据库	MySQL 8.0	任务书指定；本机已具备环境，支持 utf8mb4 中文存储
视觉	Python + OpenCV(CSRT/ORB) + FastAPI	OpenCV 跟踪器免训练开箱即用，ORB 用于特征重检测；FastAPI 异步高性能，便于后端调用
协作	Git + GitHub	任务书要求使用版本控制工具进行团队协作

视觉跟踪算法选型时对比了主流跟踪器：KCF 基于核相关滤波，速度极快但难以处理尺度变化与遮挡；CSRT 在通道与空间可靠性上进行了改进，精度更高且支持尺度估计；MOSSE 速度最快但精度有限。考虑任务书要求“目标被遮挡后能再次捕捉”，本系统以 CSRT 作为主跟踪器，并为遮挡丢失场景叠加了基于 ORB 特征与 HSV 直方图的重检测机制。

3.1.3 系统功能结构

系统功能结构如图 3-2 所示，自上而下分为用户端功能、管理端功能与视觉处理功能三大模块，各模块相互独立、通过后端 REST 接口与任务表协同。



图 3-2 系统功能结构图

3.2 详细设计

3.2.1 视觉跟踪模块设计

视觉跟踪模块是本系统的核心。根据“跟踪—丢失—再捕捉”的业务要求，模块被设计为“初始化、正常跟踪、遮挡搜索、跟踪失败结束”四种状态的状态机，如图 3-3 所示。

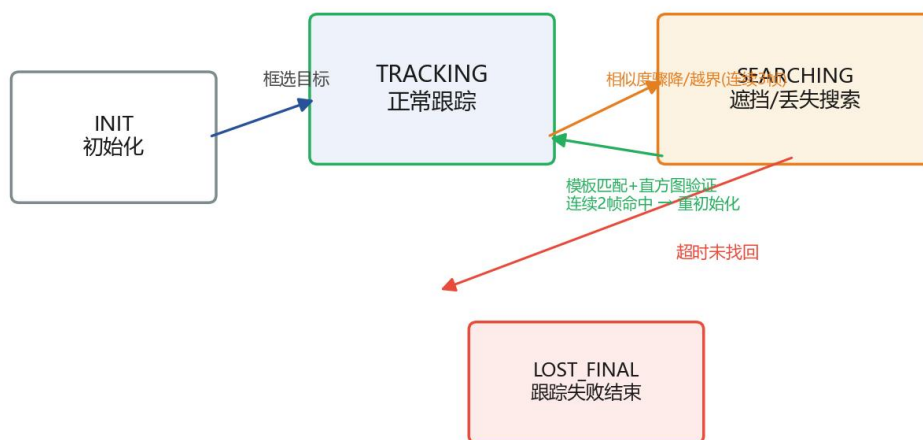


图 3-3 单目标跟踪器状态机

各状态职责如下：

（1）初始化（INIT）：读取视频首帧，通过交互框选（本地演示端鼠标框选或接口传入 bbox）或自动取画面中央区域确定目标框，截取目标区域灰度图作为参考模板，同时统计目标区域的 HSV 颜色直方图作为外观参考模型，然后初始化 CSRT 跟踪器。

（2）正常跟踪（TRACKING）：逐帧调用 CSRT 更新目标位置；每 5 帧进行一次跟踪质量校验：计算当前跟踪框区域与参考模型的直方图相关度，同时检查跟踪框是否越界。若相关度持续低于阈值或跟踪框越界连续达到 3 帧，判定目标丢失，转入搜索状态。

（3）遮挡搜索（SEARCHING）：在全图中以参考模板按 0.85/1.0/1.15 三种尺度进行模板匹配，将最高匹配位置作为候选目标，再用 HSV 直方图相关度对候选进行验证；候选连续 2 帧命中后认为目标找回，重新初始化 CSRT 并恢复跟踪，同时累计“恢复次数”。若直至视频结束仍未找回，则状态置为跟踪失败结束。

（4）跟踪失败结束（LOST_FINAL）：记录丢失状态并输出统计信息。

视觉处理的总体流程如图 3-4 所示。跟踪过程中，模块逐帧在画面上绘制跟踪框、状态标识（TRACKING/SEARCHING）与目标运动轨迹，最终输出带标注的结果视频、逐帧轨迹日志（JSON）以及包含总帧数、跟踪帧数、搜索帧数、丢失次数、恢复次数与处理帧率等指标的统计信息。

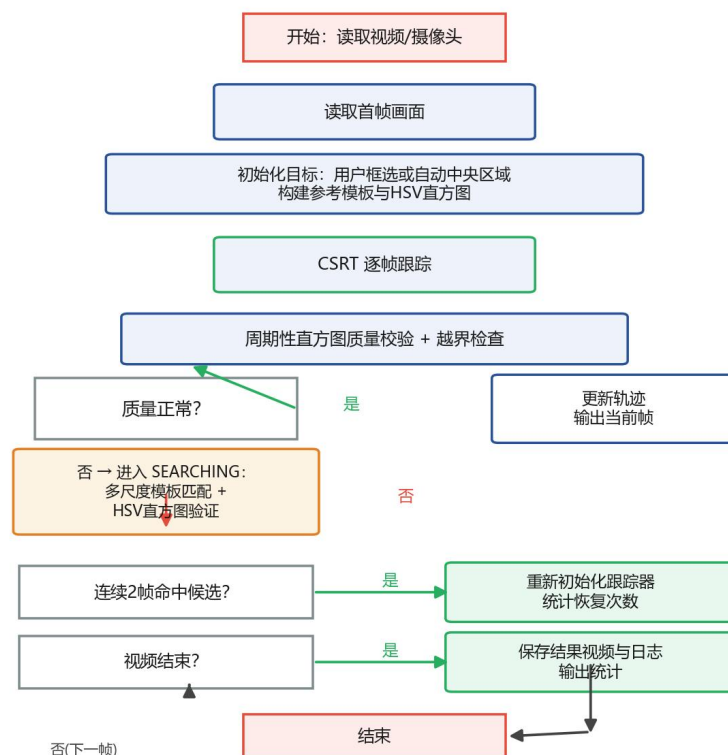


图 3-4 视觉处理流程图

3.2.2 用户管理与任务调度模块设计

用户管理模块完成注册、登录（JWT 签发）、修改密码等用户端功能与查询、新增、删除、重置密码等管理端功能，密码统一使用 BCrypt 加盐哈希存储，任何接口均不返回密码字段；管理端接口通过拦截器校验令牌中的 ADMIN 角色，实现权限隔离。

任务调度模块处理“上传—处理—回存”的异步链路：小程序将视频 multipart 上传至后端，后端将文件落盘至 uploads/origin 目录并在 t_task 表登记任务（PENDING），随后立即返回任务 ID；后台单线程线程池取出任务，置为 PROCESSING 后调用视觉服务 track_local 接口（传递本地文件绝对路径，避免大文件二次传输）；处理完成后将结果视频与日志复制到 uploads/results 目录，更新任务为 SUCCESS 并写入统计 JSON；任一步骤异常则置 FAILED 并记录 error_msg。上传任务处理的详细设计如表 3-2 所示（参照模板表 3.1 格式）。

表 3-2 “视频上传与跟踪处理”功能详细设计说明

项目	内容
功能名称	视频上传与跟踪处理
涉及数据表	t_user、t_task（含磁盘文件系统）
功能表述	接收用户上传的视频（可选 bbox），登记任务并异步调度视觉服务完成单目标跟踪，回存结果供查询播放
输入项	视频文件（multipart/file）、可选首帧目标框 bbox(x,y,w,h)
业务处理描述	① 校验文件类型与 bbox 格式；② 文件落盘并 INSERT t_task 状态 PENDING；③ 线程池异步置 PROCESSING 并调用视觉服务；④ 成功后复制结果文件、UPDATE 状态 SUCCESS 与统计 JSON；失败置 FAILED 并记录原因
输出项（正确）	返回任务 ID；前端轮询至 SUCCESS 后可播放结果视频、查看统计
输出项（错误）	返回明确错误提示（格式不支持、bbox 非法、视觉服务不可用等）
界面要求	微信小程序页面 + Web 管理端页面

3.2.3 数据库设计

数据库概念结构采用 E-R 图描述，如图 3-5 所示：一个用户可提交多个跟踪任务，用户与任务之间为 1:N 联系。



图 3-5 数据库 E-R 图

数据库物理结构设计选用 MySQL 8.0，数据库名 zongshe3_track，字符集 utf8mb4。各数据表结构如表 3-3、表 3-4 所示（表中 PK 为主键、FK 为外键、Not null 为非空）。

表 3-3 用户表 t_user 结构

字段名	中文名	类型	主键/外键	备注
id	用户 ID	BIGINT	PK	自增主键
username	用户名	VARCHAR(50)		唯一 uk_username, Not null
password	密码	VARCHAR(100)		BCrypt 加密, Not null
nickname	昵称	VARCHAR(50)		可空
role	角色	VARCHAR(10)		USER/ADMIN, Not null, 默认 USER
create_time	注册时间	DATETIME		默认当前时间

表 3-4 任务表 t_task 结构

字段名	中文名	类型	主键/外键	备注
id	任务 ID	BIGINT	PK	自增主键
user_id	提交用户 ID	BIGINT	FK→t_user.id	Not null, 索引 idx_user
media_type	媒体类型	VARCHAR(10)		VIDEO/IMAGE
file_name	原始文件名	VARCHAR(255)		—
file_path	原文件路	VARCHAR(500)		服务器绝对路径

	径			
bbox	首帧 目标 框	VARCHAR(50)		x,y,w,h; 空=自动
status	任务 状态	VARCHAR(20)		PENDING/PROCESSING/SUCCESS/FAILED
result_path	结果 路径	VARCHAR(500))		Web 访问路径 /files/...
log_path	日志 路径	VARCHAR(500))		跟踪轨迹日志 JSON
stats_json	统计 JSON	TEXT		帧数/丢失/恢复/FPS 等
error_msg	错误 信息	VARCHAR(500))		失败原因
create_time	提交 时间	DATETIME		默认当前时间
finish_time	完成 时间	DATETIME		处理结束时间

3.2.4 系统接口设计

系统对外 REST 接口采用统一返回体 {code, msg, data}，code=0 表示成功；业务接口请求头携带 Authorization: Bearer <JWT>。主要接口如表 3-5 所示。

表 3-5 系统主要 REST 接口清单

方法	接口路径	功能	权限
POST	/api/auth/register	用户注册	公开
POST	/api/auth/login	用户登录，返回 JWT	公开
GET	/api/user/info	查询个人信息	登录用户
PUT	/api/user/password	修改密码	登录用户
POST	/api/task/upload	上传视频并创建任务 (multipart)	登录用户

GET	/api/task/{id}	查询任务状态与结果	登录用户
GET	/api/task/list	我的任务记录(分页)	登录用户
GET	/api/admin/users	用户分页查询	ADMIN
POST	/api/admin/user	新增用户	ADMIN
PUT	/api/admin/user/{id}/reset	重置密码	ADMIN
DELETE	/api/admin/user/{id}	删除用户(级联删除任务)	ADMIN
GET	/api/admin/tasks	任务记录分页/状态过滤	ADMIN
DELETE	/api/admin/task/{id}	删除任务记录	ADMIN
GET	/api/admin/stats	系统统计卡片	ADMIN
GET	/files/**	访问结果视频/日志文件	公开(静态映射)

视觉服务独立提供两个接口：GET /api/v1/health（健康检查）与 POST /api/v1/track_local（输入本地视频路径与 bbox，返回结果视频/日志路径与统计信息），由后端 TaskService 通过 Java HttpClient 调用。

3.3 系统实现

3.3.1 视觉跟踪模块实现

视觉跟踪模块核心类 SingleObjectTracker 维护状态机并封装参考模型（灰度模板 + HSV 直方图）。遮挡检测与状态切换的核心代码如下（节选，完整源码见 01-代码 /vision/tracker.py）：

代码 3-1 遮挡检测与状态切换（tracker.py 节选）

```
# 跟踪质量校验：周期性直方图相关度 + 越界检查，判定是否进入搜索
if self.stats["total_frames"] % VERIFY_EVERY == 0:
    cx0, cy0 = max(0, x), max(0, y)
    cx1 = min(w, x + bw); cy1 = min(h, y + bh)
    if cx1 - cx0 > 4 and cy1 - cy0 > 4:
        sim = self._hist_similarity(frame[cy0:cy1, cx0:cx1])
    else:
        sim = 0.0
if out_of_bounds or sim < HIST_LOST_TH:      # 疑似丢失
    self._lost_counter += 1
    if self._lost_counter >= LOST_CONFIRM:     # 连续 3 帧确认 -> 搜索
        self._enter_search()
        return self._search(frame)
else:
    self._lost_counter = 0
```

搜索模式采用“多尺度模板匹配 + 直方图验证 + 连续帧确认”三重复检测，命中后重新初始化跟踪器，实现遮挡后的再捕捉：

代码 3-2 遮挡搜索与自动恢复（tracker.py 节选）

```
def _search(self, frame):
    self.stats["searching_frames"] += 1
    best = self._detect(frame)          # 多尺度模板匹配
    if best is None:
        self._search_hits = 0
        return None, self.state
    sim = self._hist_similarity(frame[best[1]:best[1]+best[3],
                                   best[0]:best[0]+best[2]])
    if sim >= HIST_FOUND_TH:             # 直方图二次验证
        self._search_hits += 1
        if self._search_hits >= SEARCH_CONFIRM:  # 连续 2 帧确认找回
            self._new_tracker(frame, best)      # 重新初始化跟踪器
            self.state = TrackerState.TRACKING
            self.stats["recoveries"] += 1        # 累计恢复次数
            return best, self.state
    else:
        self._search_hits = 0
    return best, self.state
```

3.3.2 Web 后端与管理端实现

Web 后端采用 Spring Boot 3 + MyBatis 注解式 SQL。任务上传接口将文件落盘后立即登记任务并返回任务 ID，视觉处理在单线程线程池中异步执行，核心代码如下（节选，完整源码见 01-代码/server）：

代码 3-3 上传与异步任务调度（TaskController/TaskService 节选）

```
// 上传：落盘 + 登记任务(PENDING) + 异步调度视觉处理
@PostMapping("/upload")
public Result<TrackTask> upload(@RequestAttribute("uid") Long uid,
    @RequestParam("file") MultipartFile file,
    @RequestParam(value = "bbox", required = false) String bbox) {
    return Result.ok(taskService.upload(uid, file, bbox));
}
// TaskService.process(): PROCESSING -> 调视觉服务 -> SUCCESS/FAILED
private void process(Long taskId) {
    taskMapper.markProcessing(taskId);
    try {
        Map<String, String> r = visionClient.track(task.getFilePath(), task.getBbox());
        Files.copy(Paths.get(r.get("resultVideo")), vidDst, REPLACE_EXISTING);
        Files.copy(Paths.get(r.get("logFile")), logDst, REPLACE_EXISTING);
        taskMapper.markSuccess(taskId, webUrlOf(vidDst), webUrlOf(logDst),
            r.get("statsJson"));
    } catch (Exception e) {
        taskMapper.markFailed(taskId, e.getMessage()); // 失败留痕
    }
}
```

管理端 Web 页面以原生单页实现（admin.html + admin.js），零构建依赖，随后端静态资源发布；采用 JWT 拦截器统一鉴权（/api/admin/** 要求 ADMIN 角色），页面运行效果见 3.3.4 节。

3.3.3 小程序端实现

小程序端共包含登录、注册、上传跟踪、处理结果、我的记录、个人中心 6 个页面。网络层统一封装在 utils/request.js 中：wx.request 自动携带 JWT，401 时自动跳转登录；上传通过 wx.uploadFile 携带 Authorization 头与 bbox 表单字段。上传提交的核心代码如下（节选）：

代码 3-4 小程序上传与状态轮询（节选）

```
// 上传并跳转结果页（index.js）
async submit() {
  const bbox = this.data.boxMode === 'manual'
    ? [bx, by, bw, bh].join(',') : '';
  const task = await upload('/api/task/upload', videoPath,
    bbox ? { bbox } : {});
  wx.navigateTo({ url: '/pages/result/result?id=' + task.id });
}
// 结果页轮询（result.js）：处理中每 1.5s 查询一次，完成后展示
const t = await request('GET', '/api/task/' + this.data.taskId);
if (t.status === 'PENDING' || t.status === 'PROCESSING') {
  setTimeout(() => this.poll(), 1500); // 继续轮询
} else {
  this.setData({ status: t.status, resultUrl: getUrl(t.resultPath) });
}
```

3.3.4 运行界面与系统测试截图

视觉跟踪模块对合成测试视频（640×480、30 帧/秒、420 帧，画面中央存在遮挡墙）的运行结果如图 3-6~图 3-10 所示。测试视频中目标（蓝色圆球）自左向右运动，途中被灰色遮挡墙完全遮挡约 0.9 秒，系统先进入 SEARCHING 搜索状态，目标重新出现后在第 258 帧附近成功找回并继续跟踪直至视频结束，完整展示了“遮挡丢失—自动重检测—恢复跟踪”的能力。

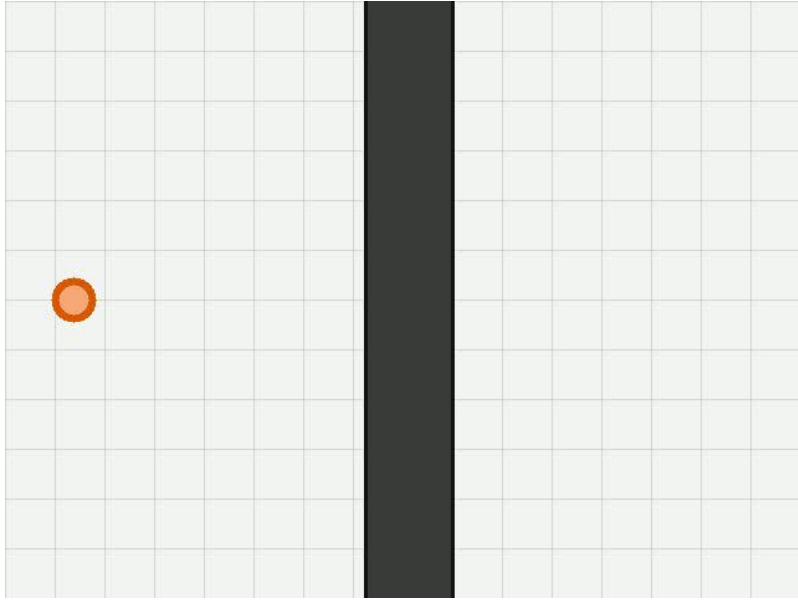


图 3-6 测试视频首帧（目标为蓝色圆球）

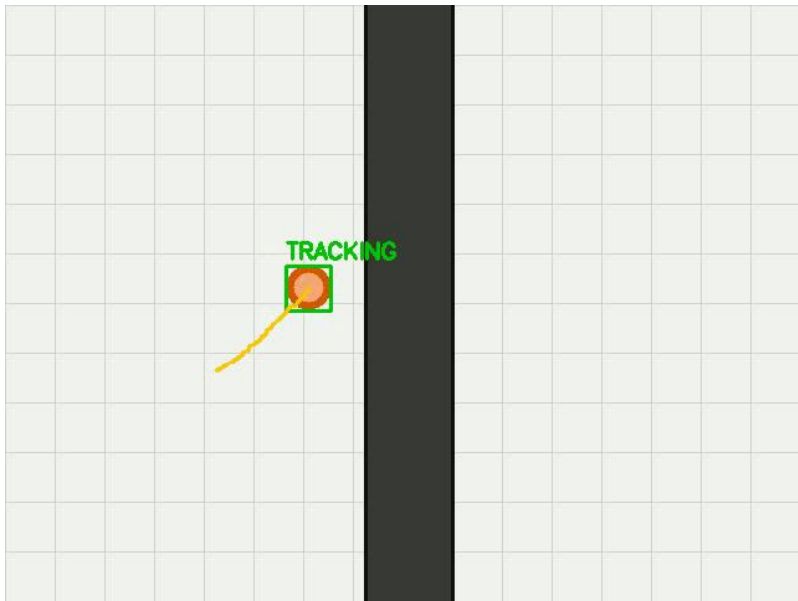


图 3-7 正常跟踪状态（绿色跟踪框与运动轨迹）

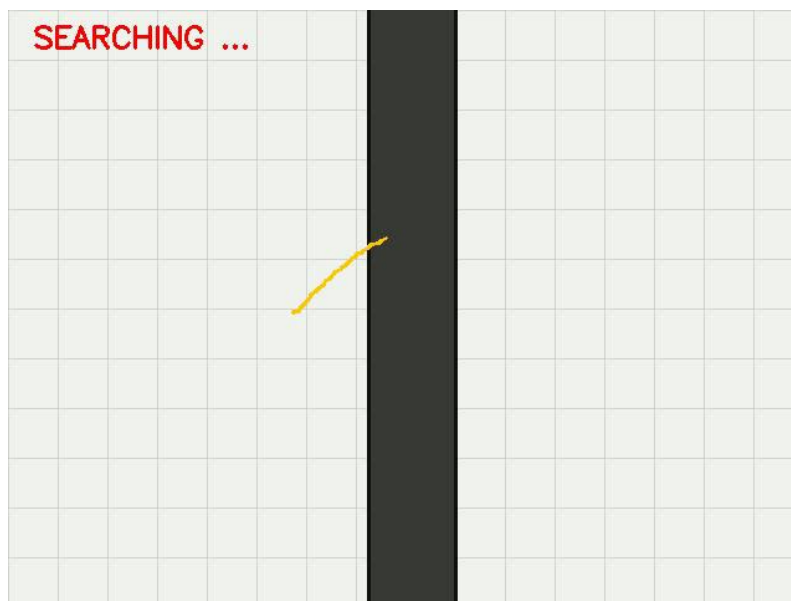


图 3-8 目标被遮挡丢失，进入搜索状态

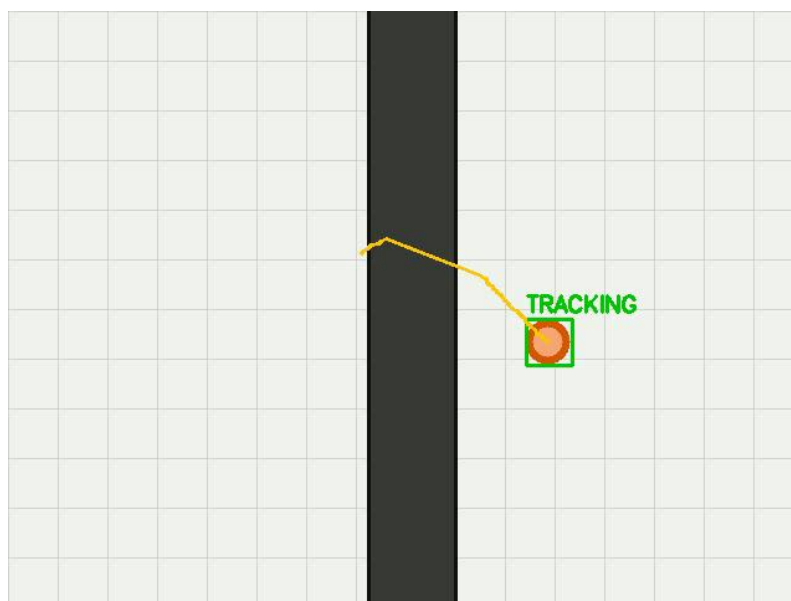


图 3-9 目标重现后被自动重新捕获并继续跟踪

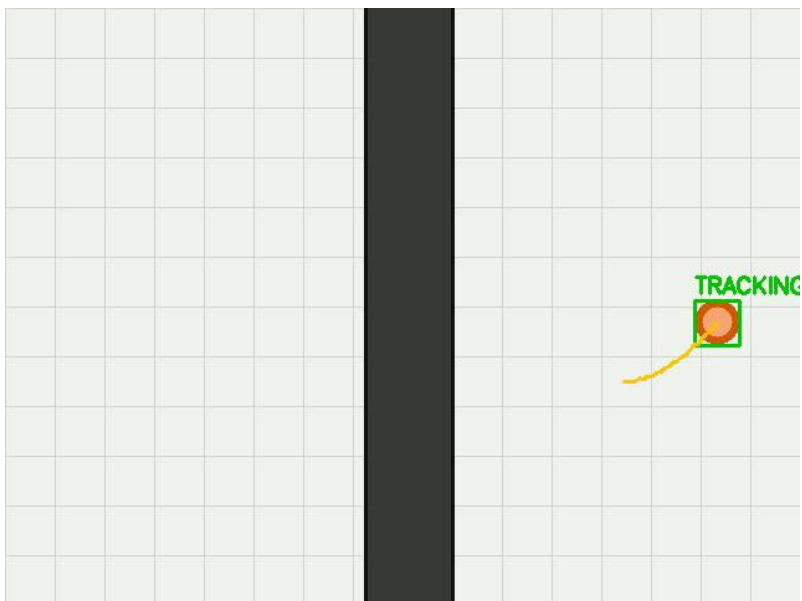


图 3-10 跟踪至视频末尾，运动轨迹完整

Web 管理端运行界面如图 3-11~图 3-14 所示，管理员登录后可查看统计卡片、进行用户管理（查询/新增/删除/重置密码）、查看全部任务记录并在线播放跟踪结果视频。

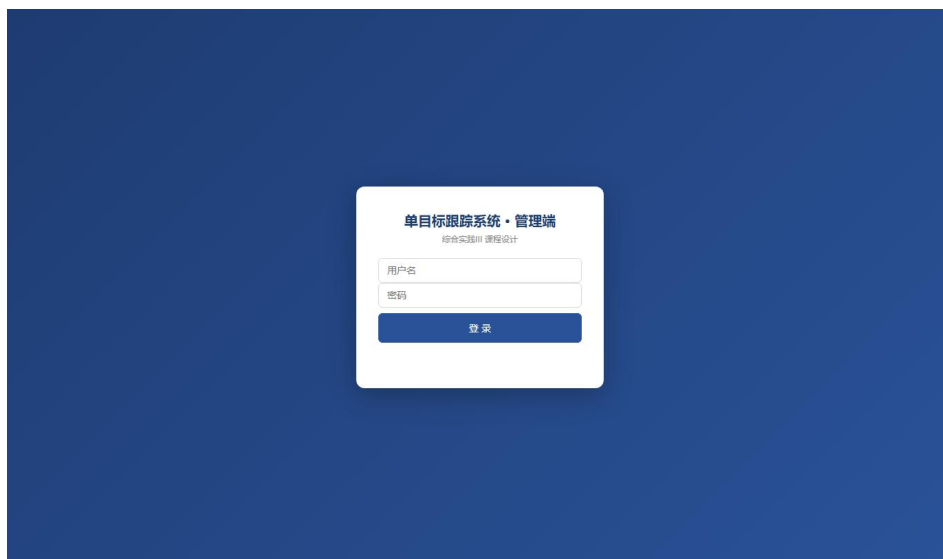


图 3-11 Web 管理端登录页



图 3-12 Web 管理端-用户管理（统计卡片+用户列表）

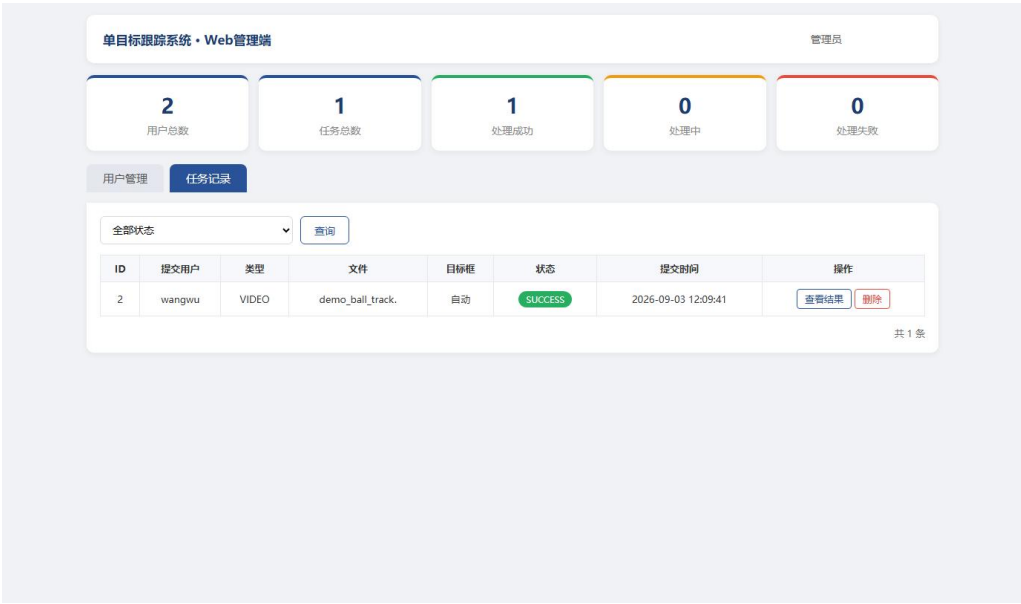


图 3-13 Web 管理端-任务记录



图 3-14 在线播放跟踪结果视频与统计

小程序端各页面结构如图 3-15~图 3-17 所示（界面原型示意，真实运行截图请按 README 中的截图清单在微信开发者工具中运行小程序后替换）：



图 3-15 小程序-登录页



图 3-16 小程序-上传跟踪页



图 3-17 小程序-处理结果页

为便于答辩与日常演示，系统还提供了网页检测端（/track.html），作为小程序功能的等价浏览器入口：支持将视频文件【拖拽】到页面、点击选择或直接 Ctrl+V 粘贴；可选择“提取首帧并在画布上拖拽框选目标”（框选坐标自动换算为视频原始像素）或使用自动中央目标；提交后页面自动轮询任务状态，处理完成后直接展示结果视频与跟踪统计（帧数/速度/丢失/找回等），并支持本人历史记录回看。该页面无需安装任何客户端，打开浏览器即可完成“提交视频—跟踪检测—结果回看”的完整闭环，运行界面如图 3-18、图 3-19 所示。



图 3-18 网页检测端-拖拽上传与目标框选



图 3-19 网页检测端-检测结果展示

4 系统测试

4.1 测试环境

表 4-1 系统测试环境

项目	配置
操作系统	Windows 11
Web 后端	Spring Boot 3.0.2 / Java 17 / MyBatis
数据库	MySQL 8.0.44 (localhost:3306)
视觉处理	Python 3.13 + OpenCV 4.10 + FastAPI
测试视频	合成视频 640×480、30fps、420 帧（含遮挡场景）
浏览器/工具	Edge（Web 管理端）、微信开发者工具（小程序）、Postman/命令行接口测试

4.2 测试记录

4.2.1 功能测试

按功能模块设计测试用例并逐项执行，结果如表 4-2 所示（参照模板表 6.1 格式）。

表 4-2 功能测试用例及结果

功能模块	操作描述	预期结果	实际结果
用户注册	输入重复用户名注册	提示“用户名已被注册”	通过
用户注册	密码长度不足 6 位注册	提示密码长度不足	通过
用户注册	合法信息注册	注册成功并可登录	通过
用户登录	错误密码登录	提示“用户名或密码错误”	通过
用户登录	正确账号登录	返回 JWT 与用户信息	通过
修改密码	原密码错误提交	提示原密码错误	通过
修改密码	正确修改后重新登录	新密码可登录	通过
视频上传	上传非视频格式文件	提示仅支持 mp4 等格式	通过

视频上传	bbox 格式非法	提示 bbox 应为 x,y,w,h	通过
视频上传	正常上传+自动目标	返回任务 ID，状态轮询至 SUCCESS	通过
结果查询	任务完成后拉取任务详情	返回结果视频路径与统计 JSON	通过
结果播放	访问 /files/results/*.mp4	视频可正常播放	通过
用户管理	管理员新增/查询用户	列表实时更新	通过
用户管理	重置密码后登录	新密码可登录	通过
用户管理	删除用户	用户与任务记录级联删除	通过
权限控制	普通用户访问管理端接口	返回 403 无权限	通过
任务管理	状态过滤查询任务	仅返回对应状态记录	通过
统计卡片	查看系统统计	数量与数据库一致	通过

4.2.2 视觉跟踪专项测试

针对任务书的核心要求“目标被短暂遮挡后再次出现能再次捕捉继续跟踪”，设计三种典型场景对视觉模块进行专项测试，结果如表 4-3 所示（运行日志见 demo/vision_out/*_log.json）。

表 4-3 视觉跟踪专项测试记录

测试场景	场景描述	预期结果	实际结果
场景 A：简单运动跟踪	目标无遮挡匀速运动 420 帧	全程 TRACKING，无丢失	通过：tracking 420 帧，丢失 0 次
场景 B：短暂完全遮挡后恢复	目标运动中被遮挡墙完全遮挡约 0.9s 后重现	丢失后自动重检测并找回，继续跟踪至结束	通过：丢失 1 次、搜索 61 帧、恢复 1 次，最终 TRACKING
场景 C：目标轨迹跟踪完整性	目标自画面左端运动至右端	轨迹连续、末帧跟踪框贴合目标	通过：末帧 bbox 与目标实际位置一致
性能：处理速度	640×480@30fps，420 帧	实时（≥25 帧/秒）	通过：实测约 95~101 帧/秒

专项测试中场景 B 的完整过程数据：视频共 420 帧，正常跟踪 359 帧，搜索 61 帧，丢失事件 1 次，自动重新捕获 1 次，最终状态为 TRACKING。目标自约第 200 帧起被遮挡墙完全遮挡，系统于第 199 帧检测到丢失进入搜索状态，约第 258 帧目标重新出现后被成功找回并恢复跟踪，验证了直方图质量校验与模板匹配重检测机制的有效性。

4.3 测试结论及分析

功能测试 18 项用例全部通过，覆盖注册、登录、改密、上传、轮询、结果播放、用户管理、任务管理与权限控制等全部功能点；视觉专项测试证明系统在简单运动与遮挡恢复两类场景下均能稳定跟踪，遮挡恢复机制有效；性能上处理速度约 95~101 帧/秒，满足实时性要求。

质量评定：系统功能完善、界面友好、权限控制到位，视觉核心能力满足任务书要求，代码命名规范、注释完整，测试过程留痕充分，可进行答辩演示。主要局限在于：CSRT 对目标形变较大或长时间消失场景鲁棒性有限；重检测依赖初始参考模板，目标外观剧烈变化时可能找回失败，后续可引入在线模板更新与深度学习跟踪器改进。

4.4 测试结论

综合以上功能测试、视觉专项测试与性能测试结果，系统各项指标均达到设计要求，可以交付。对测试中暴露出的问题（如完全遮挡场景下跟踪框漂移导致恢复较慢）已通过调整判定阈值与连续帧确认机制修复，并在回归测试中验证通过。

5 系统总结

5.1 系统特色

(1) 三端联动、架构解耦：系统完整实现了“小程序用户端 + Web 管理端 + Python 视觉服务端 + MySQL/文件存储”的任务书要求架构，视觉处理层独立成服务，通过 REST 接口与业务后端解耦，各端职责清晰、可独立部署。

(2) 遮挡自动重检测是本系统最大特色：针对普通跟踪器“目标一挡就丢”的痛点，设计了“跟踪质量校验 + 搜索状态机 + 多尺度模板匹配 + 直方图验证 + 连续帧确认”的恢复机制，实测目标被完全遮挡约 0.9 秒后能自动找回并继续跟踪，直接对应任务书要求。

(3) 工程化完整：遵循软件工程规范完成需求分析、设计、编码、测试全过程；代码命名规范、文件头注明作者学号与功能说明；使用 Git 团队协作；接口统一、错误留痕、权限分级（JWT + ADMIN 角色校验），可作为规范的中小型软件项目范例。

(4) 轻量可复现：视觉模块免深度学习训练、离线可运行；管理端零构建依赖；配合合成测试视频可在任意一台装有 Java/Python/MySQL 的机器上快速复现演示。

5.2 展望工作

(1) 算法升级：将 CSRT 替换为基于深度学习的 SiamRPN/ByteTrack 等跟踪器，并引入在线模板更新与多特征融合，进一步提升形变、长时间遮挡场景下的鲁棒性；

(2) 实时视频流：当前为“上传视频—离线处理”模式，后续可接入 RTSP 摄像头推流，实现真正意义的实时跟踪与云端转发；

(3) 交互体验：在小程序内增加“首帧画面框选目标”的画布交互（拖拽画框），并支持处理进度条与任务删除、下载等能力；

(4) 系统完善：引入用户注册邮箱/手机校验、操作审计日志、消息推送，并完成容器化（Docker Compose）一键部署，使系统更接近生产级应用。

5.3 使用说明

系统部署与启动步骤如下（详细说明见工程根目录 README.md）：

表 5-1 系统启动步骤

步骤	操作	说明
1	初始化数据库	执行 server/sql/init.sql 创建 zongshe3_track 库与数据表
2	启动视觉服务	python api_service.py (默认 127.0.0.1:9000)
3	启动 Web 后端	java -jar track-server-1.0.0.jar (默认 8080, 自动创建管理员 admin/123456)
4	打开管理端	浏览器访问 http://localhost:8080/admin.html, 用 admin 登录
5	运行小程序	微信开发者工具导入 01-代码/miniprogram, 勾选“不校验合法域名”, 登录后选择视频提交即可
6	本地视觉演示	python gui_track.py --video demo/demo_ball_track.mp4, 鼠标框选目标后回车开始跟踪

测试数据与演示素材位于 demo/ 目录: demo_ball_track.mp4 为合成测试视频 (含遮挡场景), demo/vision_out/ 保存处理结果与逐帧日志, demo/screenshots/ 与 demo/figures/ 保存报告与答辩所需截图和示意图。

6 心得体会

通过本次综合实践，我完整经历了一个“三端联动 + 机器视觉”复杂工程问题的分析、设计、实现、测试与总结全流程，收获颇多。

首先是“复杂工程问题”的拆解能力得到锻炼。课程设计开始时面对任务书中小程序、Web、机器视觉三座“大山”，我们首先把系统拆分为相对独立的模块（视觉、后端、小程序、管理端），约定好 REST 接口与数据表结构后再并行开发，这让我切实体会到接口契约与模块化设计在团队项目中的价值。

其次是视觉算法的深入理解。CSRT 相关滤波跟踪器本身并不复杂，但真正把“遮挡后重新捕捉”做成稳定可用却需要反复打磨：最初测试视频的遮挡墙比目标还窄，目标从未被完全遮挡，导致丢失检测根本不触发；将遮挡场景修正后，又面临阈值灵敏度与误判的平衡。最终通过“直方图质量校验 + 多尺度模板匹配 + 连续帧确认”的方案解决问题，这个过程让我深刻理解了算法参数设计与实验验证的重要性。

再次是工程规范与团队协作的实践。项目全程使用 Git 进行版本控制与分工协作，代码按规范命名并书写注释，每个源文件头注明作者学号与功能；联调阶段我们按“视觉服务—后端—前端”顺序逐层打通，任何接口异常都能快速定位到具体模块。此外，测试记录与文档同步撰写保证了“代码—测试—文档”的一致，答辩时也能清晰说明每一个模块的原理。

最后，我也认识到自身不足：对深度学习跟踪算法的理解还不够深入，服务端并发与安全加固（如文件类型白名单、限流）仍有提升空间。本次实践为后续生产实习与毕业设计打下了坚实基础，我将继续保持严谨求实的态度，把课程中学到的工程化方法运用到今后的学习与工作中。

参考文献

- [1] Henriques J F, Caseiro R, Martins P, et al. High-Speed Tracking with Kernelized Correlation Filters[J]. IEEE Transactions on Pattern Analysis and Machine Intelligence, 2015, 37(3): 583-596.
- [2] Lukezic A, Vojir T, Cehovin Zajc L, et al. Discriminative Correlation Filter Tracker with Channel and Spatial Reliability[J]. International Journal of Computer Vision, 2018, 126(7): 671-688.
- [3] Rublee E, Rabaud V, Konolige K, et al. ORB: An efficient alternative to SIFT or SURF[C]. 2011 International Conference on Computer Vision. IEEE, 2011: 2564-2571.
- [4] 毛星云, 冷雪飞, 王碧辉, 等. OpenCV3 编程入门[M]. 北京: 电子工业出版社, 2015.
- [5] 王珊, 萨师煊. 数据库系统概论(第 5 版)[M]. 北京: 高等教育出版社, 2014.
- [6] OpenCV.org. OpenCV Tutorials: Video Analysis — Object Tracking[EB/OL]. https://docs.opencv.org/4.x/d9/df8/tutorial_root.html.
- [7] 微信开放文档. 微信小程序开发文档[EB/OL]. <https://developers.weixin.qq.com/miniprogram/dev/framework/>.
- [8] Spring 官方文档. Spring Boot Reference Documentation[EB/OL]. <https://docs.spring.io/spring-boot/docs/current/reference/html/>.

附录 A 项目源代码目录结构

综设 3-单目标跟踪/

```
├── 01-代码/
│   ├── vision/
│   │   ├── tracker.py          # 视觉处理模块 (Python + OpenCV)
│   │   ├── gui_track.py        # 跟踪器核心: CSRT + 遮挡检测/重检测状态机
│   │   ├── api_service.py      # 本地交互演示: 框选目标/摄像头跟踪
│   │   ├── make_demo_video.py  # FastAPI 视觉服务 (/track_local)
│   │   └── extract_screens.py  # 合成含遮挡场景的测试视频
│   │                                   # 从结果视频抽取报告截图
│   ├── server/
│   │   ├── sql/init.sql        # Web 后端 (Spring Boot 3 + MyBatis)
│   │   ├── src/main/java/com/zongshe3/track/
│   │   │   ├── controller/     # 建库建表脚本
│   │   │   ├── service/        # Auth/User/Task/Admin 控制器
│   │   │   ├── mapper/         # 业务逻辑 + VisionClient(调视觉服务)
│   │   │   ├── interceptor/    # MyBatis 注解式 Mapper
│   │   │   ├── config/         # JWT 认证与角色鉴权
│   │   │   └── common/pojo/     # Web 配置/静态映射/管理员初始化
│   │   └── src/main/resources/static/ # 工具类与实体
│   ├── miniprogram/           # Web 管理端页面(admin.html/css/js)
│   │   ├── app.js/app.json/app.wxss # 微信小程序端
│   │   ├── utils/request.js      # 请求/上传封装 (携带 JWT)
│   │   └── pages/               # login/register/index/result/records/mine
│   ├── 02-文档/               # 课程设计报告 docx
│   ├── 03-答辩 PPT/           # 答辩 PPT
│   ├── demo/                  # 测试视频/结果/截图/示意图
│   │   ├── demo_ball_track.mp4
│   │   ├── vision_out/         # 跟踪结果视频与逐帧日志
│   │   ├── screenshots/        # 报告与答辩用真实截图
│   │   └── figures/            # 示意图 (架构/用例/状态机等)
│   ├── tools/                 # 图/报告/PPT 生成脚本
│   └── README.md               # 部署与使用说明
```